

COGNIZANT DIGITAL NURTURE PROGRAM WEEK 2 TASK

1)E-commerce Platform :

Big O Notation :

It is used to measure the performance of an algorithm. It tells us how the running time increases as the input size increases.

Best, Average and Worst Case

Linear Search

Suppose there are **n products**.

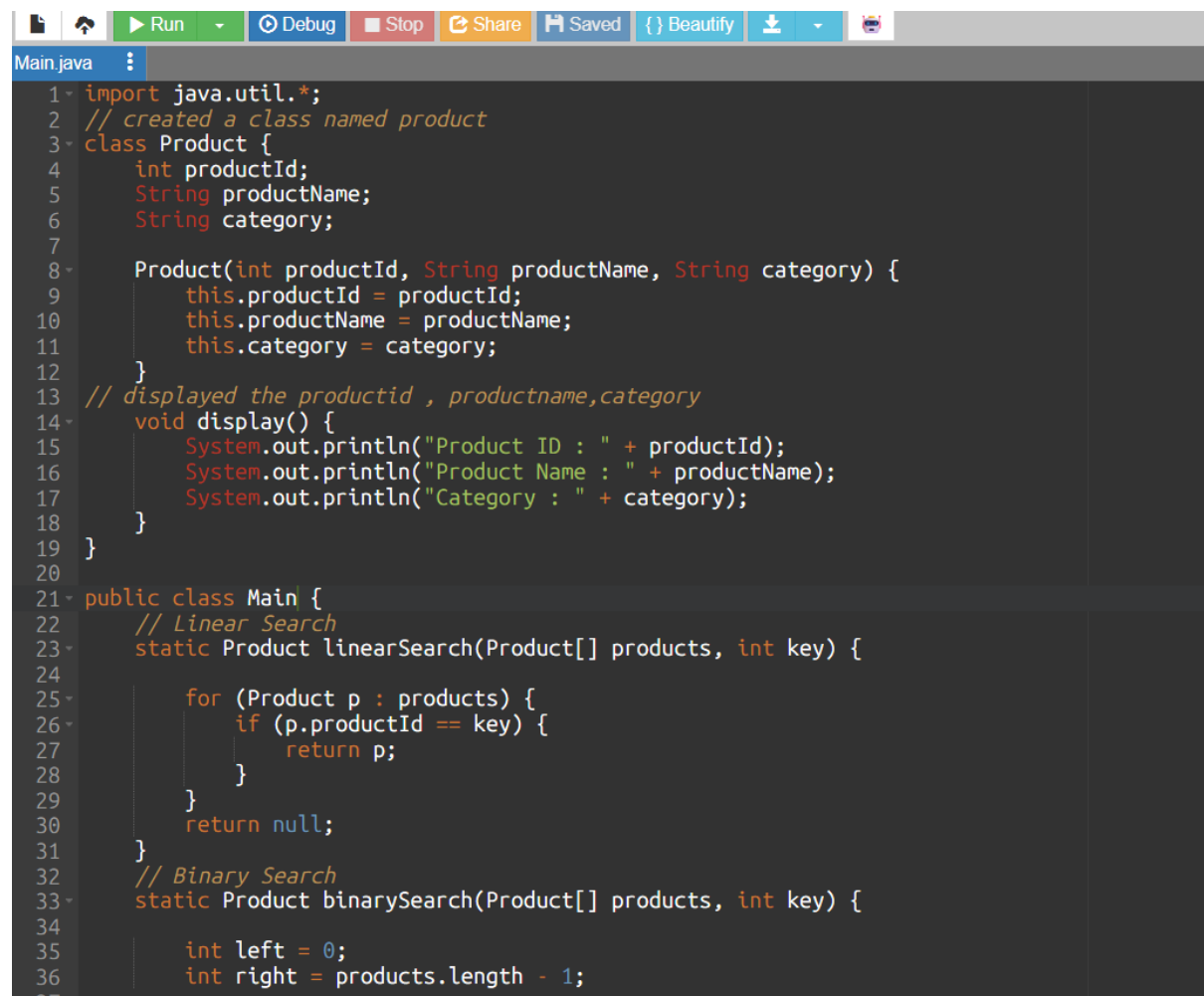
- **Best Case:** Product is found at the first position.
 - Time Complexity = **$O(1)$**
 - **Average Case:** Product is somewhere in the middle.
 - Time Complexity = **$O(n)$**
 - **Worst Case:** Product is at the last position or not present.
 - Time Complexity = **$O(n)$**
-

Binary Search

(Binary Search requires the array to be sorted.)

- **Best Case:** Product is found in the middle immediately.
 - Time Complexity = **$O(1)$**
- **Average Case:** Product is found after several divisions.
 - Time Complexity = **$O(\log n)$**
- **Worst Case:** Product is absent or found after maximum divisions.
 - Time Complexity = **$O(\log n)$**

Code :



```
1- import java.util.*;
2- // created a class named product
3- class Product {
4-     int productId;
5-     String productName;
6-     String category;
7-
8-     Product(int productId, String productName, String category) {
9-         this.productId = productId;
10-        this.productName = productName;
11-        this.category = category;
12-    }
13-    // displayed the productid , productname,category
14-    void display() {
15-        System.out.println("Product ID : " + productId);
16-        System.out.println("Product Name : " + productName);
17-        System.out.println("Category : " + category);
18-    }
19- }
20-
21- public class Main {
22-     // Linear Search
23-     static Product linearSearch(Product[] products, int key) {
24-
25-         for (Product p : products) {
26-             if (p.productId == key) {
27-                 return p;
28-             }
29-         }
30-         return null;
31-     }
32-     // Binary Search
33-     static Product binarySearch(Product[] products, int key) {
34-
35-         int left = 0;
36-         int right = products.length - 1;
```

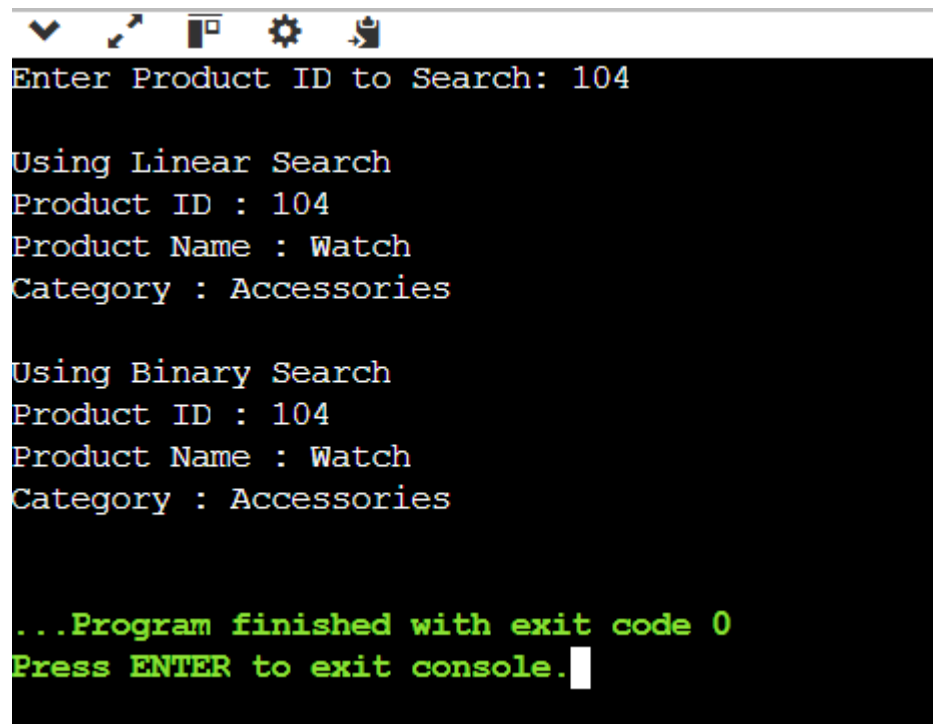
```

38 while (left <= right) {
39
40     int mid = (left + right) / 2;
41
42     if (products[mid].productId == key) {
43         return products[mid];
44     }
45
46     else if (products[mid].productId < key) {
47         left = mid + 1;
48     }
49
50     else {
51         right = mid - 1;
52     }
53 }
54
55 return null;
56 }
57 // main method
58 public static void main(String[] args) {
59
60     Product[] products = {
61         new Product(101, "Laptop", "Electronics"),
62         new Product(102, "Mobile", "Electronics"),
63         new Product(103, "Shoes", "Fashion"),
64         new Product(104, "Watch", "Accessories"),
65         new Product(105, "Book", "Education")
66     };
67
68     Scanner sc = new Scanner(System.in);
69
70     System.out.print("Enter Product ID to Search: ");
71     int key = sc.nextInt();
72
73     System.out.println("\nUsing Linear Search");

```

```
57 // main method
58 public static void main(String[] args) {
59
60     Product[] products = {
61         new Product(101, "Laptop", "Electronics"),
62         new Product(102, "Mobile", "Electronics"),
63         new Product(103, "Shoes", "Fashion"),
64         new Product(104, "Watch", "Accessories"),
65         new Product(105, "Book", "Education")
66     };
67
68     Scanner sc = new Scanner(System.in);
69
70     System.out.print("Enter Product ID to Search: ");
71     int key = sc.nextInt();
72
73     System.out.println("\nUsing Linear Search");
74
75     Product result1 = linearSearch(products, key);
76
77     if (result1 != null)
78         result1.display();
79     else
80         System.out.println("Product Not Found");
81
82     System.out.println("\nUsing Binary Search");
83
84     Product result2 = binarySearch(products, key);
85
86     if (result2 != null)
87         result2.display();
88     else
89         System.out.println("Product Not Found");
90
91     sc.close();
92 }
93 }
```

Output :

A screenshot of a terminal window with a black background and white text. The window has a title bar with standard icons (minimize, maximize, close, settings, and a terminal icon). The text inside the terminal shows the execution of a program that searches for a product by ID. It first prompts for a product ID, then shows the results for a linear search, followed by a binary search, and finally displays a completion message in green text.

```
Enter Product ID to Search: 104

Using Linear Search
Product ID : 104
Product Name : Watch
Category : Accessories

Using Binary Search
Product ID : 104
Product Name : Watch
Category : Accessories

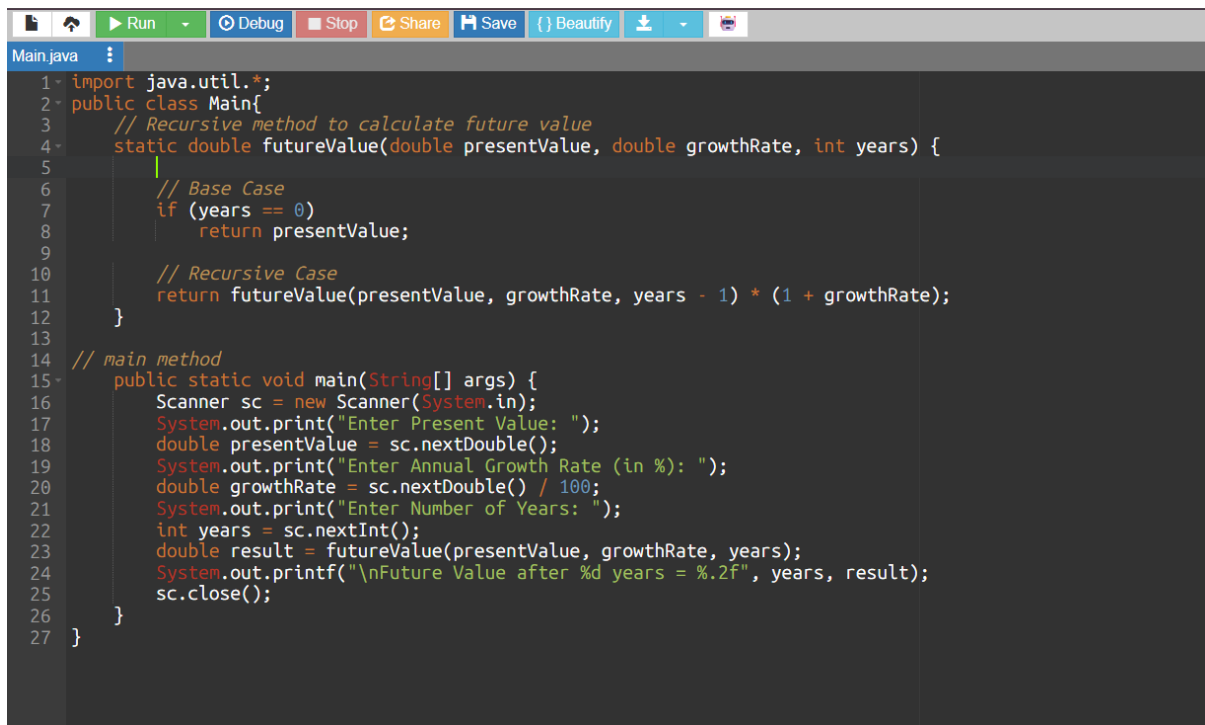
...Program finished with exit code 0
Press ENTER to exit console.
```

7) Financial Forecasting :

Recursion :

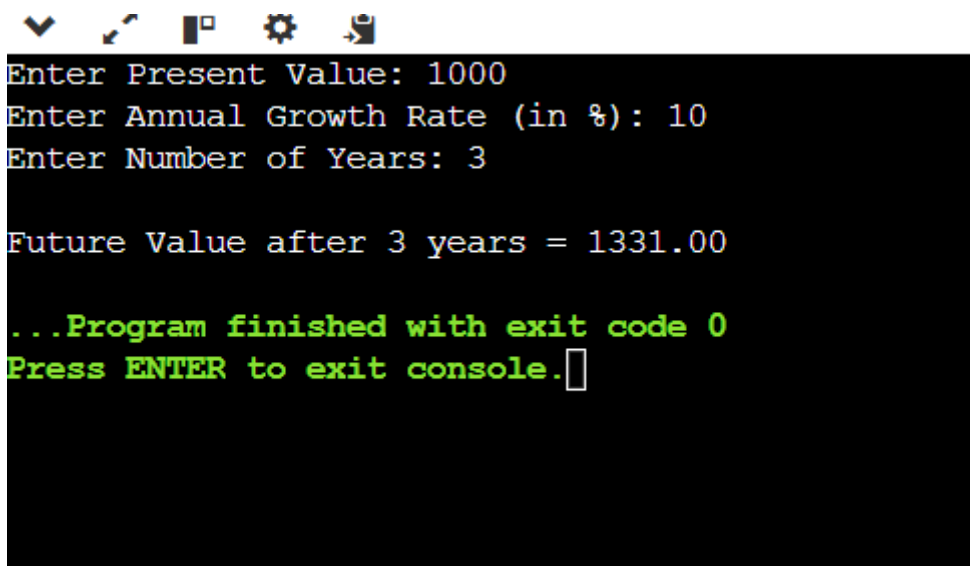
It is a programming technique where a method calls itself to solve a problem. A recursive algorithm breaks a problem into smaller subproblems until it reaches a **base case**, which stops the recursion.

Code :



```
1- import java.util.*;
2- public class Main{
3-     // Recursive method to calculate future value
4-     static double futureValue(double presentValue, double growthRate, int years) {
5-         // Base Case
6-         if (years == 0)
7-             return presentValue;
8-         // Recursive Case
9-         return futureValue(presentValue, growthRate, years - 1) * (1 + growthRate);
10-    }
11-
12-    // main method
13-    public static void main(String[] args) {
14-        Scanner sc = new Scanner(System.in);
15-        System.out.print("Enter Present Value: ");
16-        double presentValue = sc.nextDouble();
17-        System.out.print("Enter Annual Growth Rate (in %): ");
18-        double growthRate = sc.nextDouble() / 100;
19-        System.out.print("Enter Number of Years: ");
20-        int years = sc.nextInt();
21-        double result = futureValue(presentValue, growthRate, years);
22-        System.out.printf("\nFuture Value after %d years = %.2f", years, result);
23-        sc.close();
24-    }
25- }
```

Output :



```
Enter Present Value: 1000
Enter Annual Growth Rate (in %): 10
Enter Number of Years: 3

Future Value after 3 years = 1331.00

...Program finished with exit code 0
Press ENTER to exit console.
```